



Embedded Systems

Exercise 1: IDE, Threads, Semaphore, Mutex, Queue

Name:	Group:
Name:	Preparation:
Name:	Hand-in:
Mark:	
Comments:	

Plagiarism:

The students confirm with the submission of the report that all sources used have been attributed and that the report is their own work. Not permissible is the dissemination of their own report or the use of reports from other students.

1. Introduction and Aim

This exercise will introduce you to the STM32CubeIDE. The exercise is split into seven parts.

1. Flashing and toggling a LED on a bare metal program.
2. Flashing and toggling a LED using ThreadX
3. Serial communication using a UART, observing priorities
4. Observing programs using TraceX
5. Critical sections – Mutex
6. Critical sections for a shared resource – with and without a Queue
7. Thread priorities & preemption

2. Reporting

This lab exercise does not require a written report. However, the some of the exercises have to be demonstrated during or at the end of the lab session. These exercises are marked with the label **Demo** in the subtitle.

3. Preparation

Each lab exercise requires a preparation. Please look through section 4 and prepare your source code. Information about the ThreadX API can be found in the lecture notes or on github:

<https://github.com/eclipse-threadx/rtos-docs/blob/main/rtos-docs/threadx/chapter4.md>

4. Exercise

Information about the board and the programming of it, can be found in the Learning Management System (LMS → Moodle).

Note: It is useful to write these programs in different projects. Some parts of the earlier exercises may be used in later ones.

4.1. Flashing an LED – Bare Metal

Set up a project and toggle the LED LD1 (green LED next to the black button). The LED LD1 is connected to the pin PI1 of the STM32F746 microcontroller (pin D13 on CN7).

The LED LD1 should flash visibly. Observe the LED pin with an oscilloscope.

4.2. Toggle an LED – Bare Metal (Demo)

Extend the project in 4.1 to toggle the LED LD1 only when a button is pressed (blue user button). I.e. each press of the button is changing the state of the LED LD1. The button is connected to pin PI11 of the STM32F746 and pulled low via a resistor.

4.3. Flashing an LED – RTOS

Set up a new project with RTOS ThreadX support and repeat the exercise of 4.1. Use one thread to toggle the LED LD1. Use `tx_thread_sleep()` to suspend the thread between the toggle commands. The LED LD1 should flash visibly.

4.4. Toggle an LED – RTOS (Demo)

This exercise introduces semaphores and interrupts. The aim is to toggle the LED LD1 whenever the user button is pressed. This time the LED LD1 is toggled in a thread which is suspended, waiting for a semaphore to be woken up. The semaphore is raised in the callback function of the external interrupt (EXTI) of pin PI11 (connected to the user button).

4.5. Serial connection (Demo)

Set up a new project to communicate serially with the PC. Using ThreadX and the UART interface establish a connection with the PC and monitor the send data with a terminal program.

There are two possible ways to establish a serial connection to the PC using the UART:

- Using a RS232-to-USB bridge (here use USART6)
- Using the Virtual Comm Port (VCP) of the ST-Link-v2.1 debugger connection (USART1)

Both work the same in software, except for addressing the correct USART.

RS232-to-USB-Bridge only: The UART communication is to be implemented using USART6 which uses the pins PC6 (Tx) and PC7 (Rx) on the STM32F746. These two pins are also connected to the Arduino shield connector. Specifically, the receive pin (RX) PC6 is connected to D1 on connector CN4. The transmit pin (Tx) PC7 is connected to D0 on CN4. In order to use the USART6 for serial communication with the PC a **RS232-to-USB bridge** is required.



Figure 1: Arduino connector CN4

- Create a thread (priority 15, no timeslice) and use this thread to send data to the USART6/USART1. The thread should send a string when it starts and afterwards periodically every 400ms. Monitor the Transmission in a terminal program (HTerm, RealTerm, or similar).
- Add a second thread (priority 15, no timeslice) and use it to toggle a LED every 40ms. Use pin PI1 (pin D13 on CN7). Observe the LED pin with an oscilloscope.
- Increase the length of the string (part a)) to 40 characters or longer. Observe the LED pin with an oscilloscope.
- Increase the priority of the LED thread to 10. Observe the LED pin with an oscilloscope.

Report the effect of the priority on the LED (oscilloscope).

- Optional: Change the UART thread to use the interrupt driven HAL library function for serial transmission. Use a semaphore to signal the UART thread that the transmission has been completed. Observe the effect on the LED (oscilloscope) with the settings under b) and d).

4.6.TraceX

Set up TraceX for your serial communication program and observe the performance within TraceX. Vary the parameters of the programs (priority, sleep time, etc) to observe the differences.

4.7.Critical sections

In this section the use and the effect of mutual exclusion is observed. Two threads with four activities each are run concurrently. The section is split in two parts, with and without a Mutex.

Create a project with two threads (Speedy thread and Slow thread) and enable TraceX support. Furthermore, the USART1 (ST-Link VCP) or USART6 (RS232-to-USB bridge) is needed (with interrupt).

Figure 2 and Figure 3 show the activities in the two threads.

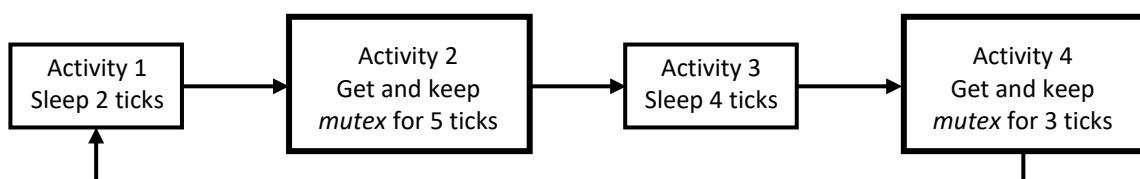


Figure 2: Speedy thread (priority = 5)

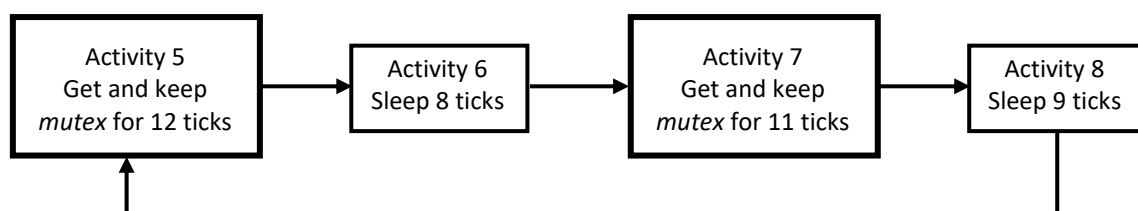


Figure 3: Slow thread (priority = 15)

4.7.1. Thread runtime

In this first part, implement the two threads **without** the use of the mutex. For the each activity the thread is suspended by the number of ticks (`tx_thread_sleep()`). At the end of each cycle print the duration of the cycle in ticks using the USART.

Run the program in debug mode, pause and load the trace file into TraceX. Try to understand how the program schedules the threads. Save a screen shot or similar.

4.7.2. Thread runtime & Critical Sections (Demo)

Now insert the Mutex at the relevant place of each thread.

The first part to declare as global entities the two threads, the mutex and the two thread stacks.

```
TX_MUTEX my_mutex;
```

The mutex is created, taken and released using:

```
Tx_mutex_create(&my_mutex, "my_mutex", TX_NO_INHERIT);  
Tx_mutex_get(&my_mutex, TX_WAIT_FOREVER);  
Tx_mutex_put(&my_mutex);
```

At the end of each cycle print the duration of the cycle in ticks using the USART.

Run the program in debug mode, pause and load the trace file into TraceX. Try to understand how the program schedules the threads. Save a screen shot or similar.

4.7.3. Critical section – USART (Demo)

The program in 4.7.2 uses two threads each calling the USART transmit function. This works seemingly ok in this setup. However, the USART is a shared resource and should not be addressed by multiple threads directly. One way to avoid corruption is to use a Semaphore/Mutex to protect the shared resource. However, this might delay the thread cycle. Another way is to use a single thread to access the USART with the other two threads passing the messages to this UART thread which in turn is the only thread accessing the USART.

Add a UART thread that reads (a) message(s) from a global variable and transmits this message to the PC via the UART.

This might require some effort to get this fully and satisfactorily working, so do not spent too much time to optimize this.

4.8. Queues (Demo)

Improve the Program from 4.7.3 by adding a queue rather than using global variables to pass messages to the UART thread.

The idea is that each thread (slow and speedy) passes a message into the queue and that the UART thread reads these messages and out puts these through the USART.

Note:

Queue creation and usage is shown below:

```
TX_QUEUE my_queue;           // Queue Control Block
static CHAR queue_memory[24]; // Queue memory (example size)

/* Create a queue */
tx_queue_create(&my_queue, "My_Queue", 3, &queue_memory, sizeof(queue_memory));

CHAR rx_msg[24];              // Local memory for message (example size)

/* Send message through queue */
tx_queue_send(&my_queue, rx_msg, TX_NO_WAIT);

/* Receive message through queue */
tx_queue_receive(&my_queue, rx_msg, TX_WAIT_FOREVER);
```

Implement and test the program. Observe the performance using TraceX.

4.9.Thread priorities & Preemption (Demo)

Create a project with four threads (Thread_x, x= A, B, C, D). The project implements a producer-consumer application with the following features:

- One producer (Thread_D), three consumers (Thread_x, x=A, B, C)
- 9 messages sent, 9 messages received
- Continuous operation: Thread_D sends 3 messages (to Thread_x, x=A, B, C) 3 times. Then it relinquishes itself (tx_thread_relinquish()).

Events should be analysed using TraceX to count context switches and measure throughput. Add to each thread (A – D) a count variable to count the number of executions per thread.

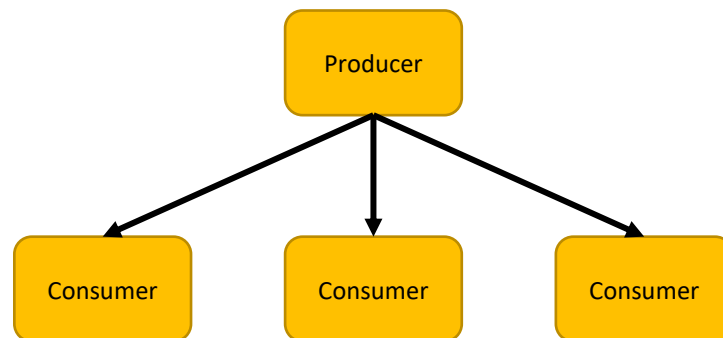


Figure 4: Producer-Consumer application

Implement the program and give the threads initially all the same priority. Test if it compiles and runs.

The effect of different priorities and the pre-emption will be explored in this part of the laboratory exercise. Use the project from your preparation and conduct three experiments.

- Case 1 – Preemptive Scheduling
- Case 2 – Round-Robin Scheduling
- Case 3 – Preemption-Threshold Scheduling

The assignment of the priority and priority-threshold is shown in Figure 5 for each thread.

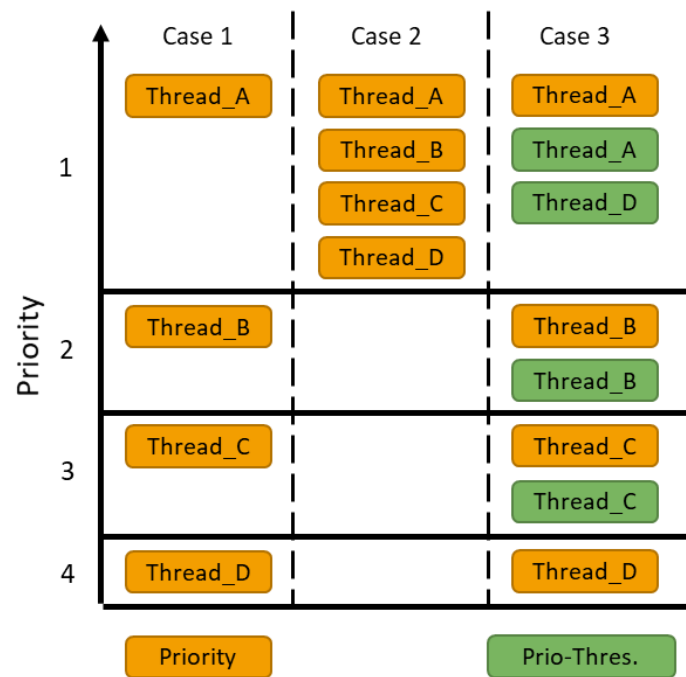


Figure 5: Priorities and Priority-Threshold

Run and analyse each case by looking at the counters, and by using TraceX. Measure the throughput and count the context switches of a cycle for each case.